

Framework DevSecOps Intelligent pour la sécurité continue

Conception et industrialisation d'une chaîne CI/CD sécurisée de bout en bout, augmentée d'une couche d'intelligence artificielle locale pour la priorisation des vulnérabilités selon leur **exploitabilité réelle**.

GitLab CI

GitLeaks

Trivy

Semgrep

Checkov

Hadolint

Syft · SBOM

Cosign

DefectDojo

Ollama · qwen2.5

EPSS · CISA KEV

Grafana

Slack

Argo CD

k3s

Oracle Cloud ARM

— Sommaire

Ce document décrit l'intégralité du framework : son architecture, chacun de ses composants, les décisions de conception qui les justifient, et les limites identifiées.

1	Présentation générale	Problématique, objectifs, application cible
2	Architecture globale	Vue d'ensemble et flux de bout en bout
3	Infrastructure	Oracle Cloud ARM, Docker, k3s, runner
4	Le pipeline CI/CD	Les 10 étapes détaillées une à une
5	Inventaire des outils	Rôle, catégorie et licence de chaque outil
6	DefectDojo — agrégation ASPM	Corrélation, déduplication, cycle de vie
7	La couche d'intelligence artificielle	Le cœur du projet : agent de triage hybride
8	La gate de sécurité	Blocage sur priorité réelle, pas sur CVSS brut
9	Chaîne d'approvisionnement	SBOM CycloneDX et signature Cosign
10	Déploiement GitOps	Argo CD, k3s et durcissement des pods
11	Observabilité et alerting	Grafana et notifications Slack
12	Résultats	Ce que le framework produit concrètement
13	Limites et perspectives	Analyse critique honnête et axes d'évolution
A	Annexes	Commandes d'exploitation et variables

■ Comment lire ce document

Encadrés bleus

Décisions de conception et justifications techniques.

Encadrés ambre

Difficultés rencontrées et correctifs appliqués.

Encadrés rouges

Limites connues et risques résiduels assumés.

1 Présentation générale

1.1 — Problématique

L'intégration d'outils de sécurité dans une chaîne CI/CD est aujourd'hui bien documentée : il suffit d'ajouter des scanners. Le problème apparaît juste après.

Sur une application web réelle de taille modeste, la chaîne complète de scanners produit **plusieurs centaines de findings** en une seule exécution. Cette masse est ingérable manuellement, et elle est surtout **mal hiérarchisée** : les outils classent par sévérité théorique (CVSS), une note attribuée en laboratoire qui ignore totalement le contexte d'exécution et la probabilité réelle d'exploitation.

LE PROBLÈME CENTRAL

Une vulnérabilité **CVSS 9.8** jamais exploitée dans la nature est traitée avec la même urgence qu'une **CVSS 6.5** activement utilisée par des rançongiciels. Résultat : les équipes se noient, désactivent les gates, et la sécurité devient un théâtre de conformité. C'est ce que la littérature appelle la *fatigue d'alertes*.

1.2 — Objectifs

Objectif 1 — Automatiser

Couvrir l'ensemble des surfaces d'attaque (code, dépendances, secrets, configuration, image, chaîne d'approvisionnement) sans intervention humaine, à chaque commit.

Objectif 2 — Centraliser

Agréger tous les rapports hétérogènes dans un référentiel unique (ASPM) avec déduplication et suivi du cycle de vie de chaque vulnérabilité.

Objectif 3 — Prioriser *intelligemment*

Remplacer le tri par CVSS par un score d'**exploitabilité réelle** combinant EPSS, CISA KEV et une analyse contextuelle par modèle de langage local.

Objectif 4 — Bloquer et déployer

Interrompre la livraison sur les priorités réelles uniquement, puis déployer via GitOps sur un cluster durci.

1.3 — L'application cible : flint_commerce

Le framework est démontré sur `flint_commerce`, une application de commerce en ligne **MERN** (MongoDB, Express, React, Node.js). Ce choix est délibéré : il s'agit d'une application réelle, non expurgée, contenant de véritables défauts — dépendances obsolètes, secrets historiquement commités, absence de durcissement.

DÉCISION DE CONCEPTION

Le dépôt est maintenu **privé**. Il contient des secrets réellement commités dans l'historique Git, conservés *intentionnellement* comme matériel de démonstration pour Gitleaks. Toute publication publique exigerait au préalable la révocation et la rotation de ces identifiants.

COMPOSANT	TECHNOLOGIE	RÔLE DANS LA DÉMONSTRATION
client	React + Vite, servi par Nginx	Surface front, build multi-étapes, image non-root
server	Express / Node.js 20	API, dépendances vulnérables, secrets d'environnement
base	MongoDB 7	Persistance, stockage in-cluster sur PVC

PORTABILITÉ

Le framework est **agnostique de l'application**. Aucune étape du pipeline ne dépend du code de `flint_commerce` : les scanners analysent un répertoire, un Dockerfile ou une image. Appliquer la chaîne à un autre projet se limite à copier `.gitlab-ci.yml` et à définir les variables d'environnement.

2 Architecture globale

Le framework s'organise en six plans successifs. Chaque plan consomme la sortie du précédent et l'enrichit — la donnée brute (un rapport de scanner) devient progressivement une décision (déployer ou bloquer).

2.1 — Les six plans

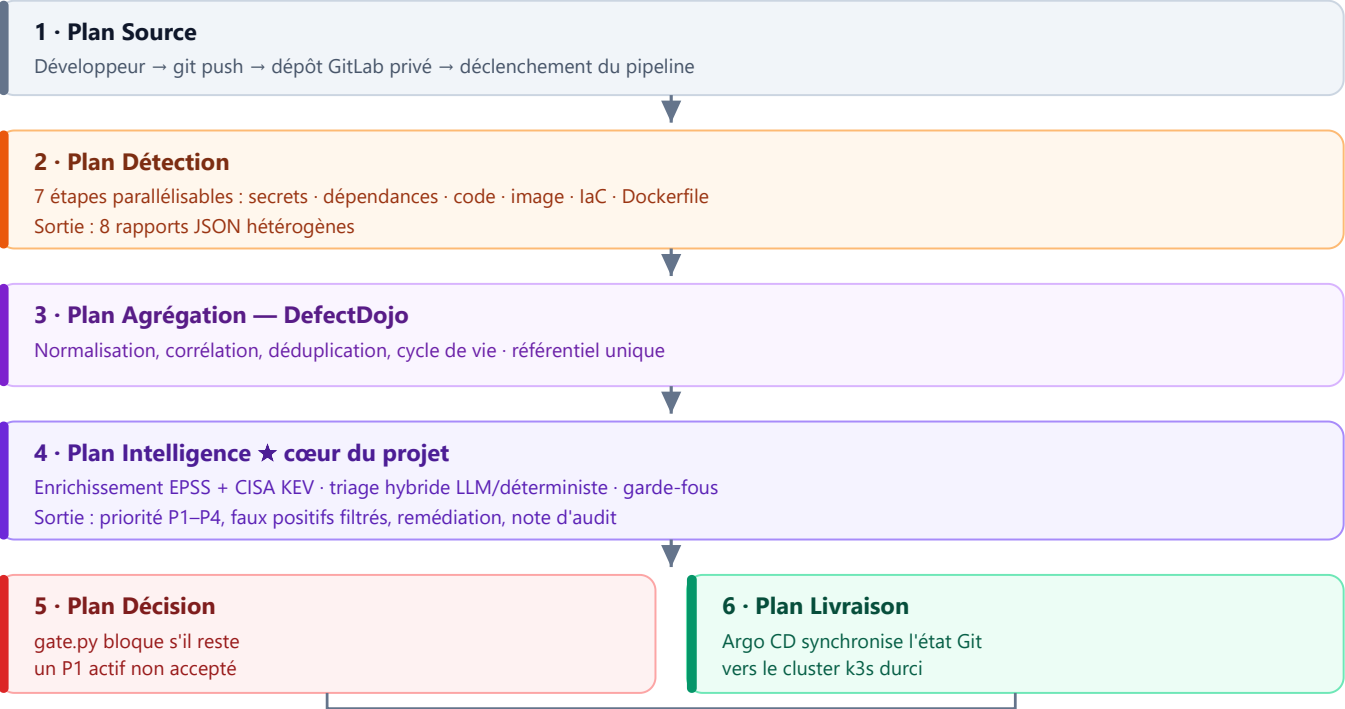


Figure 1 — Les six plans logiques du framework

2.2 — Vue d'ensemble détaillée

Le schéma ci-dessous représente l'implémentation complète, avec les composants réels et les flux qui les relient.

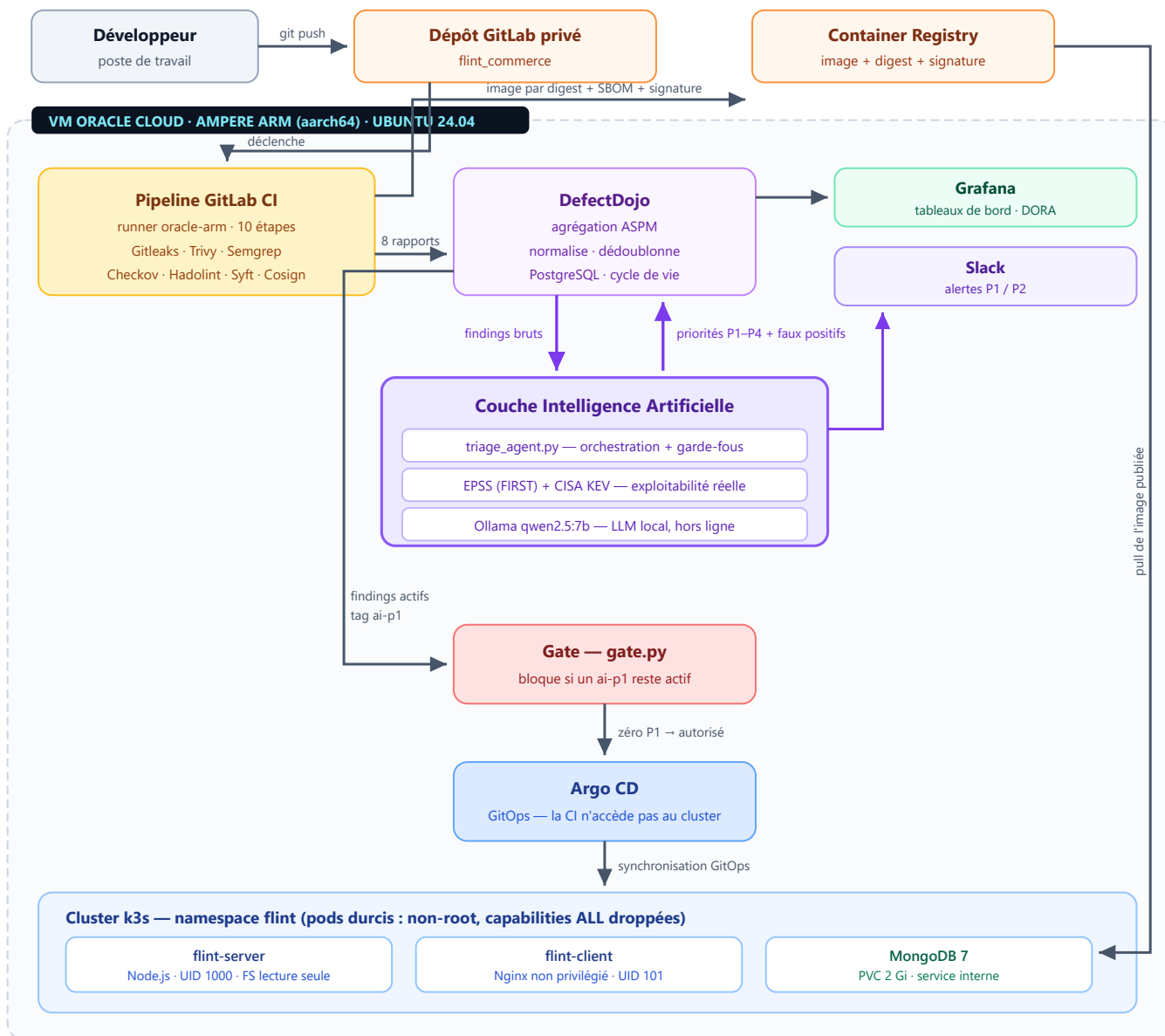


Figure 2 — Architecture d'implémentation complète

Une version haute résolution de ce schéma, éditable et enrichie des logos officiels, est disponible dans le fichier `architecture_eraser.png` livré avec le projet.

LA BOUCLE DE RÉTROACTION

Le flux notable est **bidirectionnel** entre DefectDojo et la couche IA : DefectDojo fournit les findings bruts, l'agent renvoie les priorités et les faux positifs, qui deviennent l'état de référence pour la gate. C'est cette boucle qui matérialise le terme « sécurité **continue** » — le système s'enrichit à chaque exécution plutôt que de repartir de zéro.

3 Infrastructure

L'ensemble du framework — CI, agrégation, LLM, observabilité et cluster — tourne sur une **unique machine virtuelle gratuite**. Cette contrainte a guidé la totalité des choix techniques.

3.1 — La machine hôte

CARACTÉRISTIQUE	VALEUR	CONSÉQUENCE SUR LA CONCEPTION
Fournisseur	Oracle Cloud — offre <i>Always Free</i>	Coût nul, adapté à un projet académique
Processeur	Ampere Altra — aarch64 (ARM)	Toutes les images doivent être multi-architecture
Système	Ubuntu 24.04 LTS	Python géré par le système → contrainte PEP 668
Exécution	Docker + GitLab Runner (exécuteur <code>docker</code>)	Chaque job s'exécute dans un conteneur isolé
Orchestration	k3s (Kubernetes allégé)	Empreinte mémoire compatible avec la VM

DIFFICULTÉ — L'ARCHITECTURE ARM

Une part importante de l'écosystème DevSecOps ne publie que des images `amd64`. Plusieurs blocages en ont découlé :

- DefectDojo devait être **construit localement** (`docker compose build` avant `up -d`) — un `up` direct tire les images amd64 et échoue sur `exec format error`.
- Cosign devait être épinglé au binaire `cosign-linux-arm64` v2.4.3 : la version du paquet Alpine avait supprimé l'option `--tlog-upload=false`.
- Le pipeline devait être forcé sur le runner ARM via `default: tags: [oracle-arm]`, sinon GitLab répartissait les jobs sur ses runners partagés amd64.

DIFFICULTÉ — LE DOUBLE PARE-FEU

Oracle applique **deux** couches de filtrage : les *Security Lists* côté cloud **et** les règles `iptables` de l'hôte. Ouvrir un port dans la console ne suffit donc pas. Deux pannes en ont découlé : la résolution DNS de k3s (CoreDNS) était cassée jusqu'à l'autorisation explicite des plages `10.42.0.0/16` et `10.43.0.0/16` ; et Ollama était injoignable depuis les jobs CI jusqu'à l'ouverture du port 11434 au réseau `172.17.0.0/16` (le pont Docker).

```
# Autoriser les réseaux internes k3s et le pont Docker
iptables -I INPUT -s 10.42.0.0/16 -j ACCEPT
iptables -I INPUT -s 10.43.0.0/16 -j ACCEPT
iptables -I INPUT -s 172.17.0.0/16 -p tcp --dport 11434 -j ACCEPT
netfilter-persistent save
```

3.2 — Le modèle d'exposition

Aucune interface d'administration n'est publiée sur Internet. DefectDojo, Grafana et Argo CD restent liés à la boucle locale et ne sont atteints qu'à travers un **tunnel SSH**. C'est la surface d'attaque minimale : un service d'administration compromis exposerait l'ensemble du référentiel de vulnérabilités.

```
# Depuis le poste de travail : rediriger les UI en local
ssh -L 8080:localhost:8080 # DefectDojo \
    -L 3000:localhost:3000 # Grafana \
    -L 8081:localhost:8081 # Argo CD \
    ubuntu@<ip-publique>
```

RUNNER OU CONTENEUR DE JOB ?

Distinction essentielle : le **runner** est l'agent permanent installé sur la VM ; il ne contient aucun outil de sécurité. À chaque job, il démarre un **conteneur éphémère** à partir de l'image déclarée (`aquasec/trivy`, `semgrep/semgrep` ...), y exécute le script, puis le détruit. Aucun scanner n'est jamais installé sur l'hôte — d'où la reproductibilité parfaite du pipeline.

4 Le pipeline CI/CD

Le fichier `.gitlab-ci.yml` définit **10 étapes séquentielles**. Les six premières couvrent la détection, les quatre dernières la décision et la livraison.

4.1 — Vue d'ensemble

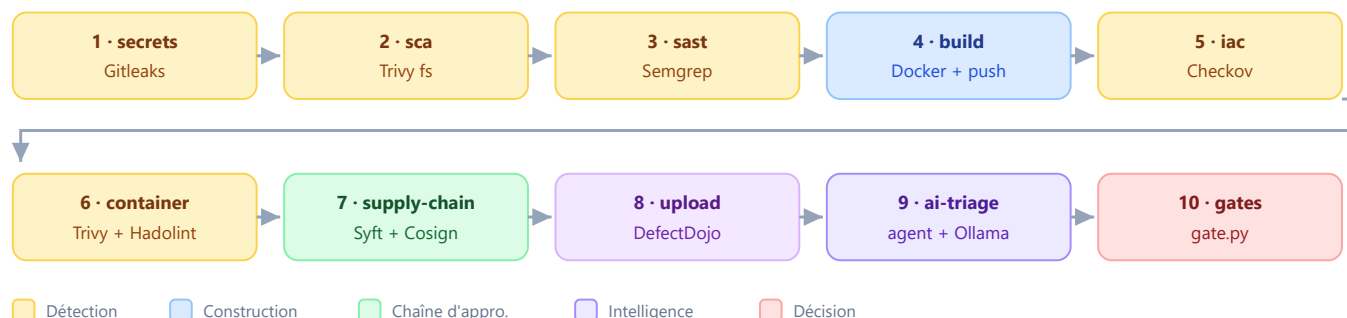


Figure 3 — Enchaînement des 10 étapes du pipeline

DÉCISION — `DEPENDENCIES: []` SUR LES JOBS DE SOURCE

Par défaut, GitLab télécharge dans chaque job *tous* les artefacts des étapes précédentes. Conséquence observée : Trivy scannait `gitleaks-report.json` et y redétectait les secrets qu'il contenait, gonflant artificiellement le total de findings. Déclarer `dependencies: []` sur chaque job qui n'analyse que les sources supprime ce bruit.

4.2 — Les étapes une par une

1

secrets — détection de secrets · Gitleaks

Image `zricethezav/gitleaks` · sortie `gitleaks-report.json`

Parcourt **l'ensemble de l'historique Git**, pas seulement l'état courant. C'est le point clé : un secret supprimé dans un commit ultérieur reste extractible depuis l'historique et demeure donc compromis. Le drapeau `--exit-code 0` empêche l'échec immédiat : la décision de blocage est déléguée à la gate finale, après le triage IA.

2

sca — analyse des dépendances · Trivy

Scanners `vuln,secret,misconfig` · sorties table + JSON

Analyse l'arbre de dépendances npm et le confronte aux bases de vulnérabilités publiques. C'est l'étape la plus productive en volume : une application Node.js tire couramment plus d'un millier de paquets transitifs. Deux formats sont produits — une table lisible dans les journaux du job, un JSON destiné à DefectDojo.

3

sast — analyse statique du code · SemgrepRègles `p/javascript` + `p/security-audit`

Recherche les motifs de code dangereux écrits par les développeurs : injections, désérialisation non sûre, cryptographie faible, validation d'entrée absente. Là où Trivy trouve les failles *importées*, Semgrep trouve les failles *écrites*.

4

build — construction et publication · Docker-in-DockerPublie dans le registre GitLab · capture le **digest** via artefact dotenv

Construit les deux images puis les pousse au registre. L'élément essentiel est la capture du **digest** (`sha256:...`) réémis dans `build.env` : les étapes suivantes référencent l'image par son empreinte cryptographique et non par son étiquette, qui est mutable.

DIFFICULTÉ — TLS AVEC DOCKER-IN-DOCKER

Le service DinD échouait sur `/certs/client/ca.pem: no such file`. Avec l'exécuteur `docker`, le volume de certificats n'est pas partagé entre le conteneur de service et celui du job. Correctif : désactiver TLS sur le réseau interne du job — isolé et éphémère — et supprimer l'alias explicite, qui entraînait en collision avec l'alias automatique.

```
variables:
  DOCKER_HOST: "tcp://docker:2375"
  DOCKER_TLS_CERTDIR: "" # TLS désactivé : réseau de job isolé
```

5

iac — configuration d'infrastructure · Checkov

Analyse les manifestes Kubernetes et la configuration déclarative

Vérifie les manifestes `k8s/` contre un référentiel de bonnes pratiques : conteneurs privilégiés, absence de limites de ressources, exécution en `root`, montages sensibles. Un repli écrit un JSON vide et valide si aucun résultat n'est produit, afin que l'étape d'import ne casse jamais.

6

container-scan — image et Dockerfile · Trivy image · Hadolint

Deux jobs parallèles sur les deux images du projet

Trivy analyse ici le **système d'exploitation de l'image** — les paquets Alpine, pas les dépendances applicatives déjà couvertes à l'étape 2. Hadolint audite la qualité des `Dockerfile` : étiquettes flottantes, exécution en root, couches inutiles.

7

supply-chain — SBOM et signature · Syft · Cosign

CycloneDX JSON, conservé 30 jours · signature par digest

Produit la nomenclature logicielle et signe cryptographiquement les images. Détaillé en **section 9**.

8

upload — import vers DefectDojo · API REST

8 imports via `/api/v2/import-scan/`

Une fonction shell `upload()` factorise les appels, avec un garde `[ -f "$file" ]` pour ignorer proprement un rapport absent. Le paramètre `test_title` est indispensable pour distinguer les trois imports Trivy (système de fichiers, image serveur, image client), qui partagent le même `scan_type`.

9

ai-triage — triage intelligent · Ollama qwen2.5:7b

Image `python:3.12-alpine` · déclenchement manuel ou planifié

Le cœur du projet, détaillé en **section 7**. Configuré en `when: manual` pour les pipelines ordinaires et en automatique pour les pipelines planifiés — l'inférence sur processeur est trop lente pour bloquer chaque commit.

10

gates — décision de livraison · gate.py

Seul job **sans** `allow_failure` — il bloque réellement

Interroge DefectDojo et échoue s'il subsiste des findings prioritaires. Détaillé en **section 8**.

DÉCISION — ALLOW_FAILURE: TRUE SUR LES SCANNERS

Tous les jobs de détection sont non bloquants. Ce n'est pas un relâchement : c'est une **séparation des responsabilités**. Les scanners *observent* et rapportent ; une seule instance — la gate, après enrichissement et triage — *décide*. Bloquer sur la sortie brute d'un scanner reviendrait à revenir au tri par CVSS que le projet cherche précisément à dépasser.

5 Inventaire des outils

L'ensemble de la chaîne repose exclusivement sur des logiciels **libres et gratuits** — un critère structurant pour la reproductibilité du projet.

ÉTAPE	OUTIL	CATÉGORIE	RÔLE	LICENCE
1	Gitleaks	Secrets	Détecte clés et jetons dans tout l'historique Git	MIT
2	Trivy	SCA	CVE des dépendances applicatives	Apache 2.0
3	Semgrep	SAST	Motifs de code vulnérables	LGPL 2.1
4	Docker / BuildKit	Construction	Images multi-étapes, publication par digest	Apache 2.0
5	Checkov	IaC	Mauvaises configurations Kubernetes	Apache 2.0
6	Trivy image	Conteneur	CVE des paquets système de l'image	Apache 2.0
6	Hadolint	Lint	Bonnes pratiques Dockerfile	GPL 3.0
7	Syft	SBOM	Nomenclature CycloneDX	Apache 2.0
7	Cosign	Signature	Signature cryptographique des images	Apache 2.0
8	DefectDojo	ASPM	Agrégation, corrélation, cycle de vie	BSD 3
9	Ollama + qwen2.5:7b	IA	Analyse contextuelle locale	MIT / Apache 2.0
9	EPSS (FIRST)	Renseignement	Probabilité d'exploitation à 30 jours	Gratuit
9	CISA KEV	Renseignement	Catalogue des failles activement exploitées	Domaine public
10	gate.py	Décision	Blocage sur priorité réelle	Développement propre
—	Grafana	Observabilité	Tableaux de bord sur PostgreSQL	AGPL 3.0
—	Slack	Alerting	Notification des priorités P1/P2	SaaS gratuit
—	Argo CD	GitOps	Synchronisation déclarative du cluster	Apache 2.0
—	k3s	Orchestration	Kubernetes allégé	Apache 2.0

5.1 — Pourquoi cette combinaison

Chaque outil couvre une surface d'attaque que les autres ne voient pas. Le tableau suivant met en évidence l'absence de redondance :

SURFACE D'ATTAQUE	OUTIL UNIQUE COMPÉTENT	CE QUE LES AUTRES MANQUENT
Secret dans l'historique Git	Gitleaks	Les autres ne lisent que l'arbre courant
Dépendance npm vulnérable	Trivy fs	Semgrep ne lit pas les <code>node_modules</code>
Faible écriture par le développeur	Semgrep	Trivy ne connaît que des CVE publiées
Paquet système de l'image	Trivy image	Trivy fs n'inspecte pas les couches de l'image
Conteneur privilégié	Checkov	Aucun scanner de code ne lit les manifestes
Provenance de l'image	Syft + Cosign	Aucun scanner ne prouve l'origine

6 DefectDojo — agrégation ASPM

Sans agrégation, huit rapports JSON dans huit formats différents restent huit silos. DefectDojo est la plateforme *Application Security Posture Management* qui les unifie en un référentiel unique et durable.

6.1 — Ce que DefectDojo apporte réellement

Normalisation

Chaque format propriétaire est converti vers un modèle *Finding* commun : titre, sévérité, composant, fichier, identifiants de vulnérabilité.

Déduplication

Un même secret Cloudinary détecté à la fois par Gitleaks et Trivy devient un seul élément suivi, et non deux tickets concurrents.

Cycle de vie

Un finding possède un état : actif, faux positif, risque accepté, corrigé. Cet état **persiste entre les exécutions** — c'est la mémoire du système.

API pilotable

L'API REST permet à l'agent IA de lire, enrichir et réécrire les findings sans intervention humaine.

6.2 — L'import depuis le pipeline

```
upload() {
  scan_type="$1"; file="$2"; title="${3:-$1}"
  [ -f "$file" ] || { echo "$file absent, skip"; return 0; }
  curl -sS -X POST "$DD_URL/api/v2/import-scan/" \
    -H "Authorization: Token $DD_API_TOKEN" \
    -F "scan_type=$scan_type" -F "file=@$file" -F "test_title=$title" \
    -F "product_name=flint_commerce" -F "engagement_name=CI/CD Pipeline" \
    -F "auto_create_context=true" -F "active=true" -F "verified=false"
}
```

PARAMÈTRE	VALEUR	POURQUOI
scan_type	Nom exact du parseur	DefectDojo rejette toute chaîne inconnue — les libellés doivent être exacts (<i>Trivy Scan</i> , <i>Gitleaks Scan</i> , <i>Semgrep JSON Report...</i>)
test_title	Libellé distinctif	Sépare les trois imports Trivy qui partagent le même scan_type
auto_create_context	true	Crée produit et engagement à la volée — pipeline autonome
verified	false	Rien n'est réputé vérifié avant le passage du triage

DIFFICULTÉ — LES NOMS DE PARSEURS

DefectDojo attend des libellés **exacts** et ne propose aucune correspondance approchée. Un `scan_type` erroné provoque un rejet silencieux : le job réussit, mais rien n'apparaît dans l'interface. Chaque nom a dû être vérifié dans la documentation officielle des parseurs.

7 La couche d'intelligence artificielle

C'est la contribution originale du projet. Le fichier `triage_agent.py` transforme un flux brut de findings en une liste de priorités fondée sur l'**exploitabilité réelle**, accompagnée de recommandations de remédiation.

7.1 — Le raisonnement

La sévérité CVSS répond à la question « *quelle serait la gravité si cette faille était exploitée ?* ». Elle ne répond jamais à « *cette faille est-elle réellement exploitée ?* ». Deux sources publiques répondent à cette seconde question :

EPSS — *Exploit Prediction Scoring System*

Publié par le FIRST. Donne, pour chaque CVE, la **probabilité qu'elle soit exploitée dans les 30 jours**. Un score de 0,94 signifie 94 % de probabilité — une information opérationnelle, absente du CVSS.

CISA KEV — *Known Exploited Vulnerabilities*

Catalogue officiel des vulnérabilités **dont l'exploitation est avérée**. Y figurer n'est pas une prédiction mais un fait constaté, avec indication d'usage par des rançongiciels.

LE PRINCIPE DIRECTEUR

Une CVE **présente au catalogue KEV** ou dont l'**EPSS dépasse 0,5** devient automatiquement **P1**, quelle que soit sa note CVSS. Inversement, une « Critique » sans exploit connu et sans probabilité d'exploitation redescend naturellement dans la file.

7.2 — Architecture de l'agent

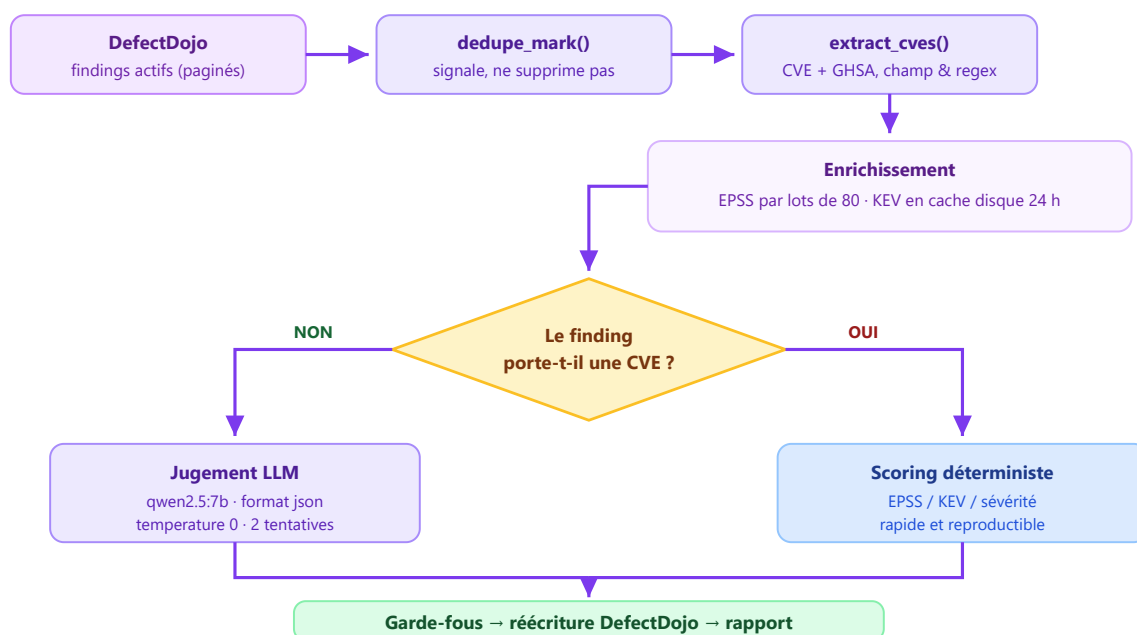


Figure 4 — Chaîne de traitement de l'agent de triage

7.3 — Le triage hybride : la décision structurante

Tous les findings ne passent pas par le modèle de langage. L'aiguillage repose sur une observation simple :

TYPE DE FINDING	TRAITEMENT	JUSTIFICATION
Avec CVE/GHSA dépendances	Déterministe	Les données objectives (EPSS, KEV, sévérité) suffisent. Faire intervenir un LLM n'ajoute aucune information et introduit du non-déterminisme.
Sans CVE secrets, SAST, IaC, Dockerfile	LLM	C'est là que le jugement contextuel a de la valeur : distinguer un secret réel d'un espace réservé exige de comprendre le contenu.

POURQUOI C'EST LE BON CHOIX

Cette répartition divise le temps de traitement par un facteur important — sur processeur uniquement, chaque appel au modèle coûte plusieurs secondes — tout en **concentrant l'IA là où elle apporte réellement quelque chose**. C'est l'inverse de l'approche naïve « tout passer au LLM », plus lente et moins fiable.

7.4 — Le prompt système

Le modèle est contraint par un schéma JSON strict et un jeu de règles ordonnées. Les paramètres d'inférence — `format: json`, `temperature: 0` — visent la reproductibilité.

```
{
  "is_false_positive": <true|false>,
  "fp_reason":        "<raison courte>",
  "risk_priority":     "<P1|P2|P3|P4>",
  "priority_score":    <entier 0-100>,
  "remediation":       "<une action concrète>"
}
```

RÈGLES (appliquées dans l'ordre) :

- Espaces réservés (FAKE, EXAMPLE, PLACEHOLDER, CHANGEME) → faux positif, P4
- kev=true OU epss ≥ 0.5 → P1, score ≥ 85
- Sinon : score selon sévérité + exploitabilité
- remediation : une action spécifique et actionnable

7.5 — Les garde-fous : la partie la plus importante

Un modèle de 7 milliards de paramètres tournant sur processeur commet des erreurs. L'ingénierie du projet consiste à **encadrer ces erreurs** pour qu'aucune ne puisse masquer une vulnérabilité réelle.

INCIDENT RÉEL — ET SON ENSEIGNEMENT

Lors d'une exécution, l'agent a classé en faux positifs **deux CVE authentiques**, puis **treize avis GHSA** concernant Nodemailer. Analyse des causes :

- Les identifiants figuraient dans le *titre* du finding, pas dans le champ structuré `vulnerability_ids` — l'agent croyait traiter un finding sans CVE et l'envoyait au LLM.
- Les avis GHSA ont été mal classés au moment où Ollama était injoignable : le repli déterministe, alors trop grossier, avait déclenché sur le mot « exemple » présent dans le texte de l'avis.

Ces faux négatifs sont le résultat le plus instructif du projet : ils démontrent expérimentalement qu'un LLM ne peut pas être l'autorité finale d'une décision de sécurité, et justifient l'architecture de garde-fous qui suit.

Garde-fou 1 — Un identifiant de vulnérabilité n'est jamais un faux positif

```
# Empêche le LLM 7B de masquer une vraie CVE
if f.get("cves") and f["verdict"]["is_false_positive"]:
    f["verdict"]["is_false_positive"] = False
    f["verdict"]["reason"] = ""
```

Garde-fou 2 — Extraction élargie des identifiants

La cause racine a été traitée : l'extraction lit désormais le champ structuré **et** applique une expression régulière au titre et à la description, couvrant CVE et GHSA.

```
pat = r"(?:CVE-\d{4}-\d{4,7}|GHSA-[0-9A-Za-z]{4}-[0-9A-Za-z]{4}-[0-9A-Za-z]{4})"
```

Garde-fou 3 — Pré-filtre déterministe des espaces réservés

Un secret factice doit être classé de façon **identique à chaque exécution**, indépendamment du non-déterminisme du modèle. Le filtre est volontairement **étroit** : il exige à la fois un indice « secret » dans le titre et un marqueur d'espace réservé — c'est précisément ce qui évite de rejouer l'incident Nodemailer.

Garde-fou 4 — Une vulnérabilité KEV n'est jamais désactivée

```
# Même si le verdict dit « faux positif », une CVE exploitée reste active
mark_fp = bool(v["is_false_positive"]) and not f.get("kev")
```

7.6 — Réécriture idempotente vers DefectDojo

L'agent réécrit ses conclusions par `PATCH` partiel — jamais par `PUT`, qui écraserait les champs non transmis.

```
patch = {"tags": new_tags,
        "false_p": mark_fp,
        "active": (not mark_fp), # un re-run RÉACTIVE un finding mal désactivé
        "verified": (not mark_fp)} # un FP ne peut pas être « vérifié »
```

DIFFICULTÉ — L'ÉCHEC SILENCIEUX

Le champ `false_p` ne persistait pas et le tableau de bord Grafana affichait obstinément zéro faux positif. La cause n'est apparue qu'en exécutant le `PATCH` à la main : DefectDojo refusait la requête avec « *False positive findings cannot be verified* ». Le code avalait alors les erreurs 4xx sans les signaler. Double correctif : poser `verified = not mark_fp`, et **remonter systématiquement les erreurs d'API** au lieu de les ignorer.

Traçabilité — la note d'audit

Chaque décision est consignée dans une note attachée au finding, préfixée par un marqueur `[AI-TRIAGE]` qui garantit l'idempotence : l'agent ne republie pas de note si elle existe déjà. Chaque verdict est donc **auditable**, avec sa source (`llm`, `deterministic` ou `fallback`), condition indispensable à l'acceptabilité d'une décision automatisée.

```
[AI-TRIAGE]
Verdict      : CONFIRMÉ
Priorité     : P1 (score 95/100)
CVE          : CVE-2024-XXXXX EPSS=94.2% KEV=OUI (exploité)
Remédiation  : Mettre à jour le composant vers la version corrigée
Source       : deterministic
-- qwen2.5:7b @ 2026-07-22 14:03:11
```

7.7 — Robustesse d'exécution

Zéro dépendance externe

Écrit intégralement avec la bibliothèque standard (`urllib`). Contourne la contrainte PEP 668 d'Ubuntu 24.04 et supprime toute dépendance à installer — l'agent tourne dans une image `python:3.12-alpine` nue.

Contrôles préalables

`preflight()` valide le jeton DefectDojo et la présence du modèle **avant** la boucle : on échoue en deux secondes plutôt qu'après vingt minutes de traitement.

Dégradation progressive

KEV indisponible, EPSS en échec ou Ollama injoignable : l'agent continue avec un repli déterministe. Il ne s'interrompt jamais.

Lecture seule par défaut

Le mode `--dry-run` est le défaut ; la réécriture exige un `--write` explicite. Aucune modification accidentelle du référentiel.

8 La gate de sécurité

Dernière étape du pipeline, et la seule qui **bloque réellement** : `gate_critical` ne porte pas `allow_failure`.

8.1 — Le critère de blocage

`gate.py` interroge DefectDojo et compte les findings qui satisfont **simultanément** quatre conditions :

CONDITION	SIGNIFICATION
<code>active = true</code>	Le finding n'a pas été corrigé
<code>false_p = false</code>	Le triage ne l'a pas classé faux positif
<code>risk_accepted = false</code>	Aucune acceptation formelle du risque
étiquette <code>ai-p1</code>	L'agent l'a jugé de priorité maximale

LA DIFFÉRENCE AVEC UNE GATE CLASSIQUE

Une gate ordinaire bloque sur « *au moins une vulnérabilité Critique* » — un critère qui produit tellement de faux blocages que les équipes finissent par la désactiver. Ici le blocage porte sur la **priorité calculée** (sévérité + EPSS + KEV + filtrage des faux positifs). Un espace réservé `sk_test_...FAKE` ne bloque donc *jamais* la livraison, tandis qu'une vulnérabilité moyenne activement exploitée la bloque.

8.2 — Choix d'implémentation

Le filtrage sur l'étiquette est réalisé **côté client**, après récupération, plutôt que par un paramètre d'URL. La syntaxe de filtrage sur les étiquettes varie selon les versions de DefectDojo (schéma *tagulous*) ; un filtre serveur mal interprété renverrait silencieusement une liste vide — c'est-à-dire une **gate qui laisse tout passer**. Le filtrage local est plus lent mais ne peut pas échouer en silence.

```
blockers = []
for f in data.get("results", []):
    if GATE_TAG in (f.get("tags") or []): # filtrage côté client : jamais silencieux
        blockers.append(f)
```

Le seuil est paramétrable par `GATE_MAX` (défaut `0` : aucune P1 tolérée). Deux voies de déblocage existent — corriger la vulnérabilité, ou l'accepter formellement dans DefectDojo. Cette seconde voie est essentielle : elle donne à la gate un **circuit de gouvernance** plutôt qu'un simple interrupteur, et évite la tentation de désactiver le contrôle.

```
🚩 Gate sécurité - produit 'flint_commerce', tag bloquant 'ai-p1', seuil=0
6 finding(s) ai-p1 actif(s) / non-FP / non-risk-accepted :
  • [High    ] Vulnérabilité critique dans une dépendance transitive
  • [Critical] Secret d'API confirmé dans l'historique Git
  ...
❌ GATE ÉCHOUÉE : 6 priorité(s) P1 non traitée(s) (seuil autorisé = 0).
→ Corriger la vulnérabilité, ou la marquer 'Risk Accepted' dans DefectDojo.
```

9 Chaîne d'approvisionnement logicielle

Depuis les compromissions de type SolarWinds, la sécurité de la chaîne d'approvisionnement est devenue une exigence à part entière — elle constitue une catégorie dédiée de l'OWASP Top 10:2025. Deux mécanismes y répondent : la nomenclature et la signature.

9.1 — SBOM : l'inventaire

Syft produit une nomenclature **CycloneDX** pour chaque image : la liste exhaustive des composants réellement embarqués, avec versions et licences.

L'UTILITÉ CONCRÈTE

À la publication d'une faille majeure dans une bibliothèque très répandue, la question opérationnelle est : « *sommes-nous concernés, et où ?* ». Sans SBOM, la réponse demande des jours d'audit. Avec un SBOM archivé par version, elle tient en une requête. C'est aussi la raison pour laquelle les SBOM sont conservés **30 jours** en artefacts.

9.2 — Signature : la provenance

Cosign signe les images avec une clé privée conservée dans les variables protégées de GitLab et chiffrée par `COSIGN_PASSWORD`.

```
cosign sign --key "$COSIGN_PRIVATE_KEY" --tlog-upload=false "$IMAGE_SERVER"
```

DÉCISION — SIGNER PAR DIGEST, PAS PAR ÉTIQUETTE

L'étape de construction exporte le digest (`sha256:...`) via un artefact `dotenv`, et c'est ce digest qui est signé. Une étiquette est mutable : signer `flint-server:abc123` ne garantit rien, car l'étiquette peut être réattribuée à une autre image. Le digest est l'empreinte cryptographique du contenu — il est immuable par construction.

DIFFICULTÉ — VERSION DE COSIGN

Le paquet Alpine installait Cosign 3.0.6, où `--tlog-upload=false` n'est plus accepté. Or le journal de transparence public Rekor n'est pas souhaitable ici : il rendrait publiques les empreintes d'un projet privé. Correctif : télécharger le binaire `cosign-linux-arm64 v2.4.3` épinglé, plutôt que de dépendre de la version du dépôt.

LIMITE MAJEURE ASSUMÉE — LA SIGNATURE N'EST PAS VÉRIFIÉE AU DÉPLOIEMENT

Dans l'état actuel, `cosign verify` s'exécute **dans le même job** que `cosign sign`. La chaîne vérifie donc sa propre signature : c'est un contrôle circulaire, qui prouve que la clé fonctionne mais **n'apporte aucune garantie au moment du déploiement**.

Le cluster k3s accepte aujourd'hui n'importe quelle image. La preuve cryptographique est produite, mais personne ne la contrôle. Le traitement de cette limite est décrit en **section 13**.

10 Déploiement GitOps

Le déploiement n'est pas réalisé par le pipeline. Argo CD observe un dépôt Git et fait converger le cluster vers l'état qui y est décrit.

10.1 — Pourquoi GitOps plutôt qu'un `kubectl apply` en CI

Déploiement piloté par la CI

- La CI détient des identifiants d'administration du cluster
- Un pipeline compromis = un cluster compromis
- Aucune détection de la dérive de configuration
- Retour arrière manuel

GitOps (Argo CD)

- **La CI n'a aucun accès au cluster**
- Argo CD tire depuis Git — flux entrant uniquement
- Dérive détectée et corrigée automatiquement
- Retour arrière = `git revert`

LE GAIN DE SÉCURITÉ

C'est une application directe du **moindre privilège**. Dans le modèle GitOps, le pipeline ne possède aucun droit sur le cluster : il ne fait que publier une image et modifier un fichier. Même totalement compromis, il ne peut pas exécuter de charge arbitraire dans le cluster. Argo CD dispose d'un accès en **lecture seule** au dépôt via un jeton de déploiement dédié.

10.2 — Durcissement des charges de travail

Les manifestes appliquent systématiquement les recommandations *Pod Security Standards* :

CONTRÔLE	VALEUR	MENACE NEUTRALISÉE
<code>runAsNonRoot</code>	<code>true</code> (UID 1000 / 101 / 999)	Évasion de conteneur vers l'hôte en root
<code>readOnlyRootFilesystem</code>	<code>true</code> sur le serveur	Dépôt de charge malveillante persistante
<code>capabilities.drop</code>	<code>["ALL"]</code>	Abus de capacités Linux privilégiées
<code>allowPrivilegeEscalation</code>	<code>false</code>	Élévation via binaires setuid
<code>seccompProfile</code>	<code>RuntimeDefault</code>	Réduction de la surface d'appels système
<code>resources.limits</code>	CPU et mémoire bornés	Déni de service par épuisement

DÉTAIL D'IMPLÉMENTATION

Le système de fichiers racine en lecture seule impose de monter un volume `emptyDir` sur `/tmp`, seul répertoire où l'application écrit. Côté client, l'image `nginx-unprivileged` écoute sur le port **8080** : un processus non privilégié ne peut pas se lier à un port inférieur à 1024.

10.3 — Les composants déployés

RESSOURCE	CONFIGURATION	EXPOSITION
Namespace <code>flint</code>	Isolation logique	—
MongoDB 7	PVC 2 Gi, stratégie <code>Recreate</code>	Service interne uniquement
flint-server	UID 1000, racine en lecture seule, sonde TCP	NodePort 30050 (cible DAST)
flint-client	UID 101, sonde HTTP <code>/healthz</code>	Service interne

11 Observabilité et alerting

Un dispositif de sécurité qu'on ne mesure pas ne se pilote pas. Deux canaux complémentaires : Grafana pour la tendance, Slack pour l'urgence.

11.1 — Grafana : les tableaux de bord

Grafana se connecte **directement à la base PostgreSQL de DefectDojo**, en lecture seule via un rôle dédié `grafana_ro` — application du moindre privilège : un tableau de bord n'a aucune raison de pouvoir écrire dans le référentiel.

Indicateurs de posture

- Findings actifs par sévérité
- Répartition des priorités P1–P4 issues du triage
- Nombre de faux positifs filtrés automatiquement
- Vulnérabilités présentes au catalogue KEV

Indicateurs de tendance

- Évolution du volume dans le temps
- Délai moyen de correction
- Taux de blocage de la gate
- Rapprochement avec les métriques DORA

DIFFICULTÉ — « DATA IS MISSING A TIME FIELD »

Les panneaux d'agrégation renvoyaient cette erreur. Cause : le format de requête était réglé sur *Time series*, qui exige une colonne temporelle. Une requête de comptage par sévérité n'en produit aucune. Correctif : passer le format à **Table** sur tous les panneaux sauf celui d'évolution temporelle.

11.2 — Slack : l'alerting

À la fin de chaque triage, l'agent publie une synthèse via un *webhook* entrant, au format Block Kit : compteurs globaux, répartition des priorités, et les cinq urgences P1/P2 les plus élevées.

DÉCISION — L'ALERTING NE PEUT JAMAIS FAIRE ÉCHOUER LE TRIAGE

La fonction `notify_slack()` intercepte toutes ses exceptions. Si le *webhook* est absent ou en erreur, un message d'avertissement est affiché et le traitement se poursuit. Une défaillance du canal de notification ne doit jamais invalider un travail d'analyse déjà accompli.

12 Résultats

12.1 — Ce que le framework produit



12.2 — Effet du triage

Sur une exécution complète, la chaîne de scanners produit plusieurs centaines de findings actifs. Le passage de l'agent réorganise cette masse :

TRANSFORMATION	MÉCANISME	BÉNÉFICE OPÉRATIONNEL
Faux positifs écartés	Pré-filtre déterministe + jugement LLM	Les espaces réservés ne bloquent plus la livraison
Doublons signalés	<code>dedupe_mark()</code> par titre et chemin	Un seul élément suivi par vulnérabilité réelle
Priorités recalculées	EPSS + KEV plutôt que CVSS brut	L'ordre de traitement reflète le risque réel
Remédiation proposée	Génération contextuelle par le LLM	Le développeur sait quoi faire, pas seulement quoi craindre
Décisions tracées	Note <code>[AI-TRIAGE]</code> par finding	Chaque verdict automatisé reste auditable

RÉSULTAT STRUCTURANT

La gate ne se déclenche plus sur du bruit. Elle bloque sur un nombre réduit de priorités **réellement exploitables** — ce qui la rend *crédible*, donc utilisable au quotidien plutôt que désactivée à la première urgence.

12.3 — Conformité aux référentiels

RÉFÉRENTIEL	ÉLÉMENT COUVERT	PAR QUEL COMPOSANT
OWASP Top 10:2025	A02 — Mauvaise configuration	Checkov, Hadolint, durcissement des pods
OWASP Top 10:2025	Chaîne d'approvisionnement	Syft (SBOM) + Cosign (signature)
OWASP Top 10:2025	Composants vulnérables	Trivy fs et Trivy image
OWASP DSOMM	Niveau 2–3 — Construction et déploiement	Pipeline automatisé, SBOM, gates
SLSA	Niveau 1–2 — Provenance	Construction scriptée, signature par digest
NIST SSDF	PW.7 / PW.8 — Revue et test du code	Semgrep, Trivy, DAST prévu

13 Limites et perspectives

Cette section recense sans complaisance les faiblesses connues du dispositif. Un travail d'ingénierie se juge autant à la lucidité de son analyse critique qu'à ses réalisations.

13.1 — Limites identifiées

L1 — LA SIGNATURE N'EST PAS CONTRÔLÉE AU DÉPLOIEMENT

Constat. Les images sont signées, mais aucun contrôle n'intervient au moment de l'admission dans le cluster. Un `kubectl apply` direct référençant une image arbitraire est accepté. La gate du pipeline est contournable puisqu'il ne s'agit que d'un job de CI.

Traitement. Déployer un contrôleur d'admission (*Kyverno* ou *OPA Gatekeeper*) portant une règle `verifyImages` adossée à la clé publique. Le cluster devient alors la frontière de confiance : toute image non signée est **rejetée à l'admission**. C'est l'évolution prioritaire.

L2 — MONGODB SANS AUTHENTIFICATION ET RÉSEAU NON SEGMENTÉ

Constat. La base est jointe par `mongodb://mongo:27017/flint`, sans identifiants, et aucune `NetworkPolicy` n'est définie. Le réseau du cluster étant plat par défaut, n'importe quel pod compromis atteint directement la base.

Traitement. Activer l'authentification MongoDB et appliquer une `NetworkPolicy` en refus par défaut, n'autorisant que le flux `flint-server → mongo` sur le port 27017.

L3 — IMAGES RÉFÉRENCÉES PAR ÉTIQUETTE MUTABLE

Les manifestes utilisent `flint-server:<SHA>`. Le pipeline calcule pourtant le digest immuable : il devrait être propagé jusqu'aux manifestes de déploiement, ce qui rendrait par ailleurs le contrôle de signature exploitable.

L4 — PERFORMANCE DE L'IA NON MESURÉE

Les faux négatifs ont été découverts *par observation*, non par une campagne de mesure. Il manque un jeu de vérité-terrain annoté permettant de calculer précision, rappel et F1 du triage, et de comparer objectivement la priorisation IA à la priorisation CVSS. C'est l'évolution la plus importante sur le plan méthodologique.

L5 — COUVERTURE DE TEST INCOMPLÈTE

Le DAST (OWASP ZAP contre le service exposé en NodePort 30050) est préparé mais non intégré au pipeline. La sécurité à l'exécution (Falco) n'est pas traitée : le dispositif détecte avant le déploiement, pas pendant l'exécution.

L6 — GESTION DES SECRETS ET INFRASTRUCTURE MANUELLE

Les secrets Kubernetes sont appliqués à la main depuis un manifeste à compléter ; un opérateur de secrets externes couplé à un coffre-fort serait la réponse industrielle. Par ailleurs la VM a été provisionnée manuellement : sa description en Terraform la rendrait reproductible et analysable par Checkov.

13.2 — Feuille de route

PRIORITÉ	ÉVOLUTION	GAIN ATTENDU
1	Contrôleur d'admission Kyverno + vérification de signature	La signature devient une garantie appliquée ; défense en profondeur
2	Campagne de mesure du triage (précision / rappel / F1)	Validation scientifique de l'apport de l'IA
3	<code>NetworkPolicy</code> et authentification MongoDB	Segmentation réseau, principe de confiance zéro
4	Intégration du DAST OWASP ZAP	Couverture des failles visibles seulement à l'exécution
5	Falco pour la détection à l'exécution	Extension du dispositif au-delà du déploiement
6	Terraform et gestion externalisée des secrets	Infrastructure reproductible et auditée

CONCLUSION

Le framework atteint son objectif : une chaîne CI/CD couvrant six surfaces d'attaque, agrégée dans un référentiel unique, et une couche d'intelligence artificielle qui substitue à la sévérité théorique une **priorisation fondée sur l'exploitabilité réelle**. Sa contribution la plus solide n'est pas d'avoir branché un modèle de langage sur des scanners — c'est d'avoir démontré, garde-fous à l'appui et incident documenté, **dans quelles limites un tel modèle peut être tenu pour fiable** au sein d'une décision de sécurité.

A Annexes

A.1 — Variables d'environnement

VARIABLE	CONSOMMATEUR	RÔLE
DD_URL	agent, gate	Adresse de DefectDojo (172.17.0.1:8080) depuis un job)
DD_API_TOKEN	agent, gate	Jeton d'API — variable GitLab masquée et protégée
OLLAMA_URL	agent	Adresse d'Ollama (172.17.0.1:11434)
OLLAMA_MODEL	agent	qwen2.5:7b
SLACK_WEBHOOK_URL	agent	Webhook d>alerting (facultatif)
GATE_MAX	gate	Nombre de P1 toléré (défaut 0)
COSIGN_PRIVATE_KEY	signature	Clé privée, chiffrée par COSIGN_PASSWORD

A.2 — Commandes d'exploitation

```
# Triage en lecture seule (aucune écriture dans DefectDojo)
python3 triage_agent.py --dry-run

# Triage complet avec réécriture
python3 triage_agent.py --write

# Démonstration rapide sur 20 findings
python3 triage_agent.py --write --limit 20

# Forcer TOUS les findings au LLM (plus lent, pour comparaison)
python3 triage_agent.py --write --llm-all

# Évaluer la gate manuellement
DD_API_TOKEN=<jeton> python3 gate.py

# État du cluster
kubectl get pods -n flint
kubectl logs -n flint deploy/flint-server --tail=50

# Accès aux interfaces via tunnel SSH
kubectl port-forward -n flint svc/flint-client 8088:8080
kubectl port-forward -n argocd svc/argocd-server 8081:443
```

A.3 — Fichiers du projet

FICHIER	RÔLE
<code>.gitlab-ci.yml</code>	Définition des 10 étapes du pipeline
<code>triage_agent.py</code>	Agent de triage IA — cœur du projet
<code>gate.py</code>	Gate de sécurité fondée sur la priorité calculée
<code>k8s/flint.yaml</code>	Manifestes Kubernetes durcis
<code>server/Dockerfile</code>	Image API multi-étapes, exécution non-root
<code>client/Dockerfile</code>	Build Vite → Nginx non privilégié
<code>server/.dockerignore</code>	Exclut <code>.env</code> — aucun secret dans l'image
<code>cosign.pub</code>	Clé publique de vérification des signatures
<code>architecture_eraser.png</code>	Schéma d'architecture

NOTE DE SÉCURITÉ

Le fichier `server/.dockerignore` constitue un contrôle à part entière : il empêche que `.env` soit copié dans l'image. Sans lui, les secrets applicatifs se retrouveraient dans une couche d'image, extractibles par simple `docker save` — indépendamment de toute variable d'environnement définie à l'exécution.